Глава 7. Дополнительные, более сложные средства языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

7.1 Индексаторы

Индексатор – разновидность свойства. С помощью индексатора можно без указания имени поля обратиться к скрытому полю-массиву объекта.

Формат объявления:

Атрибуты Спецификаторы Тип `this` [Список_параметров]

{

[`get` Метод_чтения]

[`set` Метод_записи]

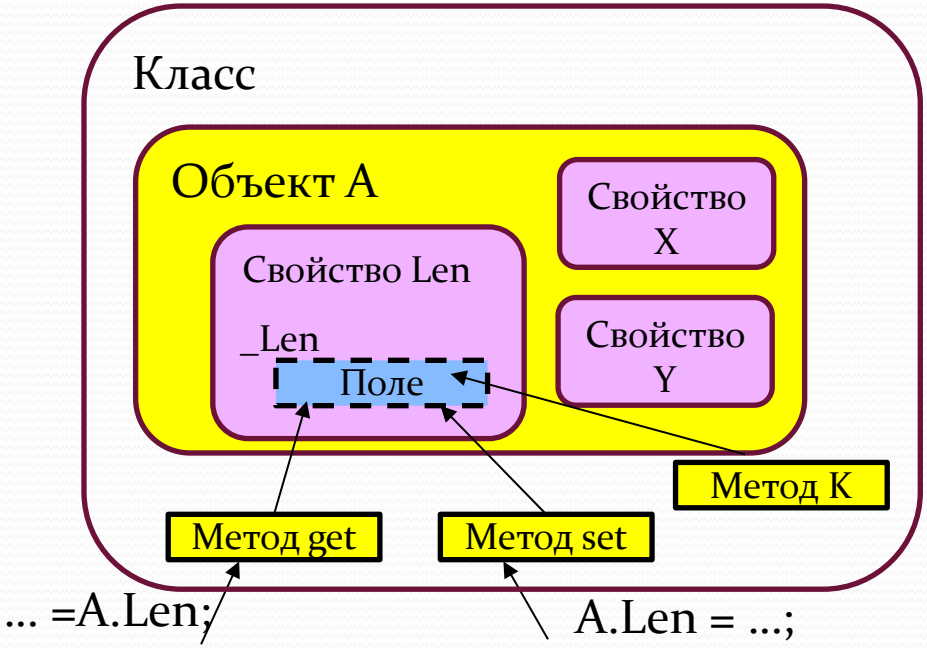
}

Спецификатор обычно **`public`**. Могут отсутствовать get или set, тогда доступ по чтению или записи недоступен.

Список параметров может включать произвольное количество индексов. Соответственно в классе может быть объявлен многомерный массив или функция с соответствующим количеством параметров.

Свойства и индексаторы (условно)

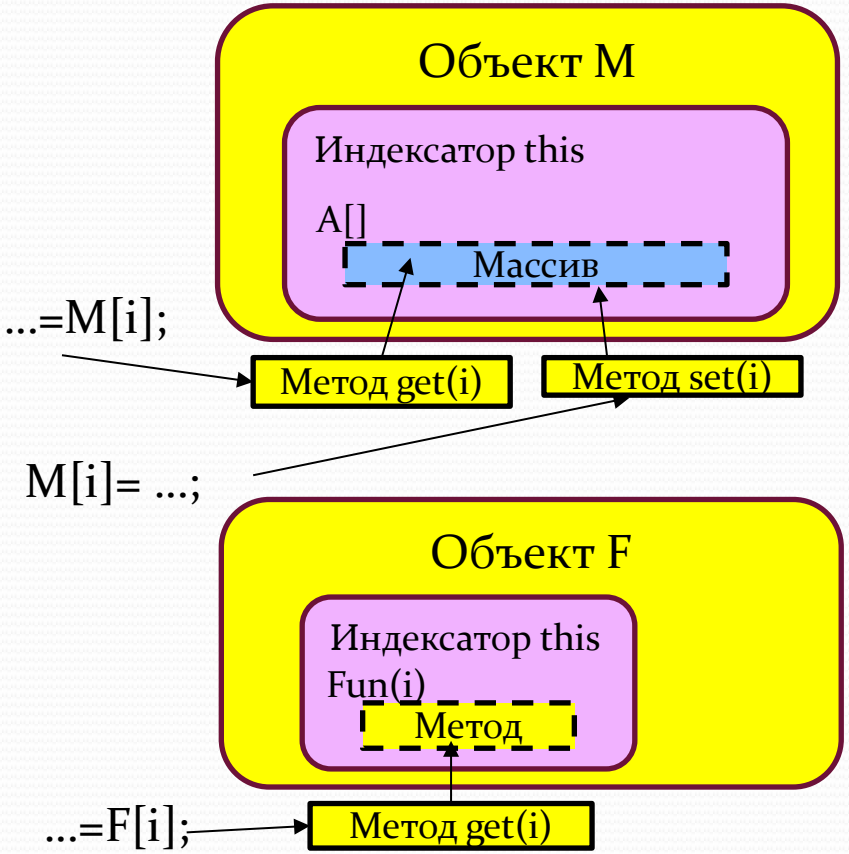
Свойства:



Объявление:
property Len {set;get;}

Обращение:
A.Len = 25; // A.setLen(25);

Индексаторы:



Объявление:
this[i] {set ...; get...;}

Обращение:
M[i] = 37; // M.set(i);

Пример

Ex7.01

```
using System;
namespace Ex7_01
{
    class SafeArray
    {
        public bool error = false;   // признак ошибки
        int[] a;                     // массив
        int length;                  // поле свойства Размер массива
        public SafeArray( int size )   // конструктор
        {
            a = new int[size];
            length = size;
        }
        public int Length   // свойство Размер массива только для чтения
        {
            get { return length;}
        }
```

Пример (продолжение)

```
public int this[int i]    // методы доступа
{
    get
    {
        if(i >= 0 && i < length) return a[i];
        else { error = true; return 0; }
    }
    set
    {
        if(i >= 0 && i < length && value >= 0 &&
            value <= 10 ) a[i] = value;
        else error = true;
     }
  }
}
```

Пример (окончание)

```
class Program
{
    static void Main()
    {
        int n = 10;
        SafeArray sa = new SafeArray(n);
        for(int i=0;i<n;++i)
        {
            sa[i] = i*2;  // обращение к массиву внутри объекта
            Console.Write("sa[{0}] = {1}",i,sa[i]);
        }
        if (sa.error)
            Console.Write("\nОбнаружены ошибки!");
    }
}
}
```

MonoDevelop External Console

```
sa[0] = 0 sa[1] = 2 sa[2] = 4 sa[3] = 6 sa[4] = 8 sa[5] = 10 sa[6] = 0 sa[7] = 0
 sa[8] = 0 sa[9] = 0
Обнаружены ошибки!
Press any key to continue...
```

7.3 Перегрузка операций
7.3.1 Перегрузка унарных операций

Можно перегрузить в классе следующие унарные операции:

`+  -  !  ~  ++  --`

Синтаксис объявления унарной операции:

`public static Тип_результата> operator@(Параметр)`

где @ - переопределяемая унарная операция;

параметр – объект класса, в котором переопределяется операция;

тип результата:

для **`+,-,!,~  -`** – произвольный тип;

для ++, -- – класс, в котором переопределяется операция.

Изменять объект-параметр нельзя! Если необходимо вернуть измененное значение объекта, то следует посредством **`new`** создавать новый объект, которых будет возвращаться в качестве результата оператором **`return`**.

Пример

Фрагмент вставляется в описание класса SafeArray:

Ex7.02

```
public static SafeArray operator ++( SafeArray x )
{
    SafeArray temp = new SafeArray(x.length);
    for(int i = 0;i < x.length; ++i)
        temp[i] = ++x.a[i];
    return temp;
}
```

Вызов:

```
sa++;
```

MonoDevelop External Console

```
sa[0] = 0 sa[1] = 1 sa[2] = 2 sa[3] = 3 sa[4] = 4 sa[5] = 5 sa[6] = 6 sa[7] = 7
sa[8] = 8 sa[9] = 9
sa[0] = 1 sa[1] = 2 sa[2] = 3 sa[3] = 4 sa[4] = 5 sa[5] = 6 sa[6] = 7 sa[7] = 8
sa[8] = 9 sa[9] = 10
Press any key to continue...
```

Перегрузка операций инкремента и декремента

```
using System;

class vector {
  int x,  y;
  public vector(int a, int b) {
    x = a;  y = b;
  }
   public void displayVector() {
      Console.WriteLine("({0}, {1})", x, y);
  }
   public static vector operator++ (vector v1) {
    v1.x++;
    v1.y++;
    return v1;
  }
}
```

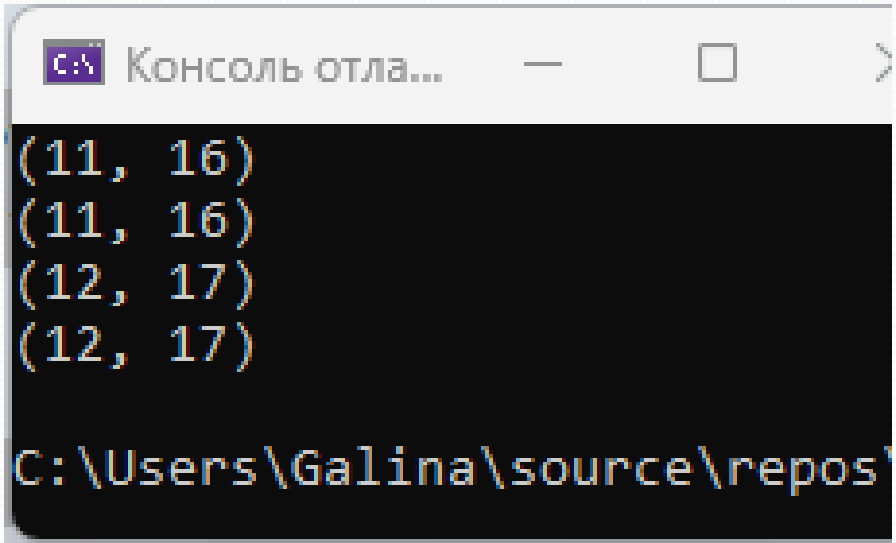
Перегрузка ++

```
class Program {
  static void Main(string[] args) {
    vector v1 = new vector(10, 15);
    vector v2;

    v2 = ++v1;
    v1.displayVector();
    v2.displayVector();

    v2 = v1++;
    v1.displayVector();
    v2.displayVector();
  }
}
```

В Windows и Linux:



7.3.2 Перегрузка бинарных операций

Можно перегрузить в классе следующие бинарные операции:

+ - * / % & | ^ << >> == != > < >= <=

Синтаксис объявления бинарной операции:

public static Тип_результата operator@(Параметр1, Параметр2)

Хотя бы один из параметров обязательно должен быть объектом.

Пример:

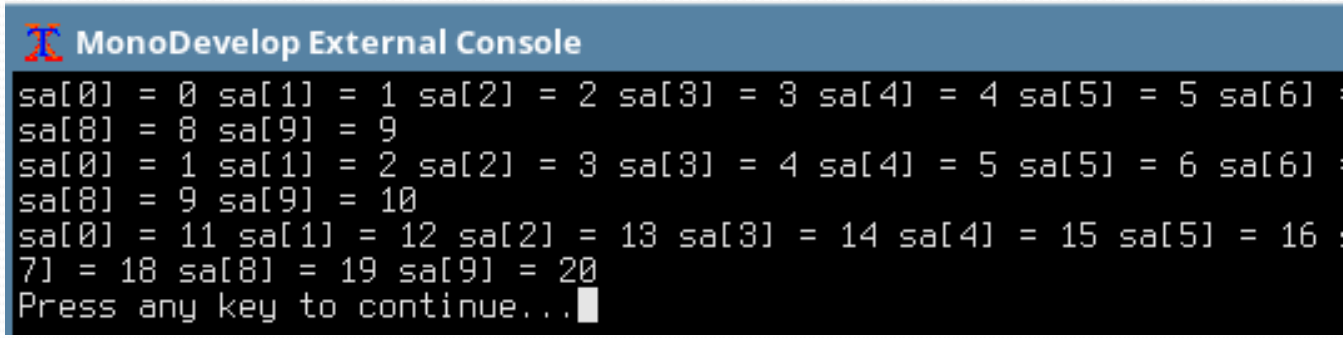
Фрагмент вставляется в описание класса SafeArray:

Ex7.02

```
public static SafeArray operator + (int x,SafeArray y)
{
    SafeArray temp = new SafeArray(y.length);
    for(int i = 0;i < y.length; ++i ) temp[i] = x+a.y[i];
    return temp;
}
```

Вызов:

```
    sa = 10 + sa;
```



7.4 Операции преобразования типа

Операции преобразования типа обеспечивают возможность явного и неявного преобразования между пользовательскими типами данных, не связанными отношением наследования. Один из типов должен быть классом.

Неявное преобразование выполняется автоматически:

1) при присваивании объекта переменной целевого типа;

2) при использовании объекта в выражении, содержащем переменные целевого типа;

3) при передаче объекта в метод на место параметра целевого типа.

Явное преобразование выполняется при явном указании операции приведения.

Программист может объявить эти преобразования для своих типов.

Синтаксис объявления операции:

`implicit operator` Тип (Параметр) // неявное преобразование

`explicit operator` Тип (Параметр) // явное преобразование

Пример

Ex7.03

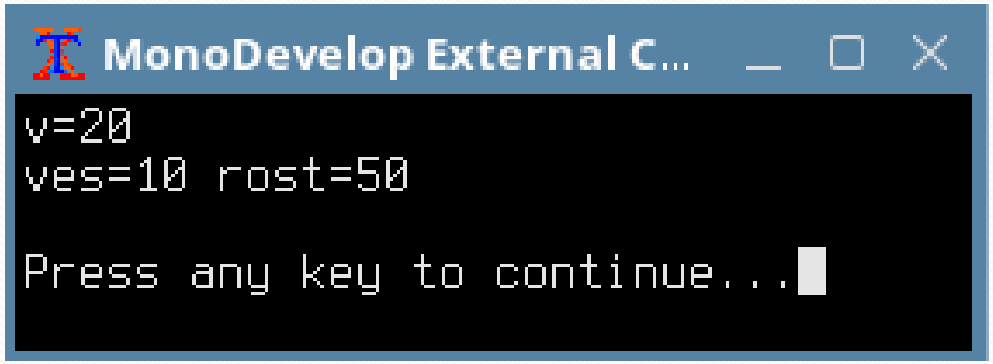
```
using System;
namespace Ex7_03
{
  class Dog
  {
     public int ves,rost;
     public Dog(int _ves,int _rost)
     { ves  =_ves; rost = _rost; }
     public static implicit operator int(Dog d)
     {  return d.ves; }
     public static explicit operator Dog(int r)
     {  return new Dog(10,r); }
     public void Print(){
        Console.WriteLine("ves={0} rost={1}",ves,rost);}
  }
```

Неявное преобразование – без потери значимости

Явное преобразование

Пример (окончание)

```
class Program
{
    static void Main()
    {
        Dog pit = new Dog (20, 50);
        int v = pit;                    // неявное преобразование
        Console.WriteLine("v = {0}",v);
        Dog bul = (Dog) pit.rost;// явное преобразование
        bul.Print();
    }
}
}
```



7.5 Деструкторы - финализаторы

Вызывается *сборщиком мусора* перед удалением объекта из памяти.

Синтаксис деструктора:

[Атрибуты] [extern] ~Имя_класса() Блок

Компилируется в финализатор:

```
protected override void Finalize()
{
    try
    {
        // здесь идут инструкции деструктора
    }
    finally
    {
        base.Finalize();
    }
}
```

В отличие от деструктора финализатор вызывается не в момент завершения работы с объектом, а в момент, когда работает сборщик мусора!

7.6 Делегаты. Лямбда-выражения

Делегат – объект, который представляет собой ссылку на метод:

[**Атрибуты**] [**Спецификаторы**] **`delegate`** **Тип Имя (Параметры);**

Когда компилятор C# обрабатывает тип делегата, он автоматически генерирует запечатанный (sealed) класс, унаследованный от **System.MulticastDelegate**. Этот класс (в сочетании с его базовым классом System.Delegate) предоставляет необходимую инфраструктуру для делегата, чтобы хранить список методов, подлежащих вызову.

Инициализируются делегаты именами методов или лямбда-выражениями.

Лямбда-выражение представляет собой упрощенную запись метода:

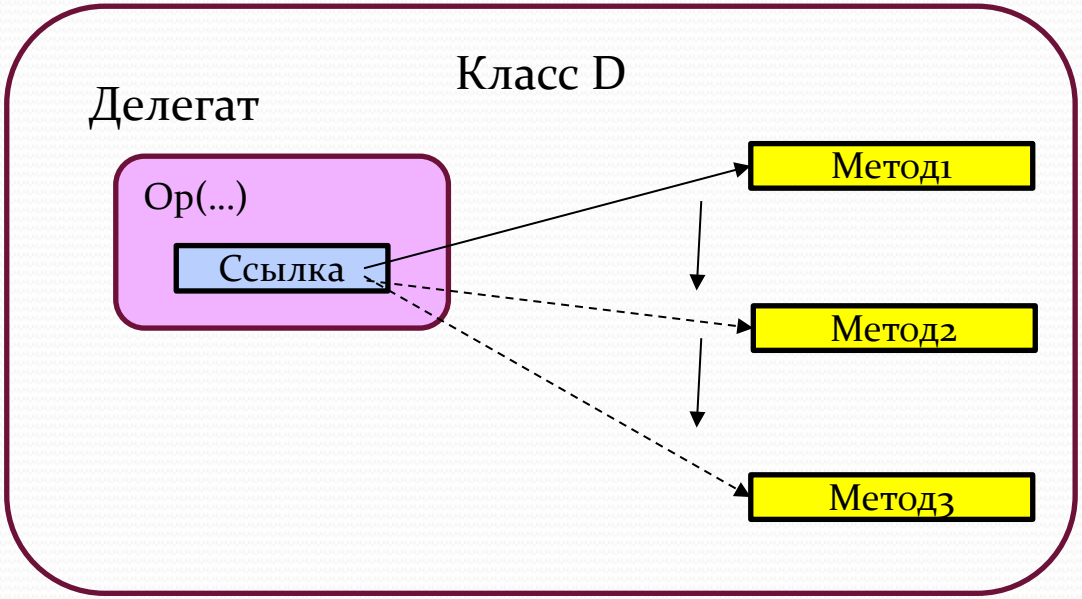
[**Тип Имя**] [(][**Входные_параметры**][)] => [{]**Операторы**[}];

- если параметров нет, то указываются пустые скобки,
- если параметр единственный, то скобки не обязательны,
- если оператор единственный, то фигурные скобки не обязательны,
- если имя не указано, то лямбда-выражение анонимно.

Примеры:

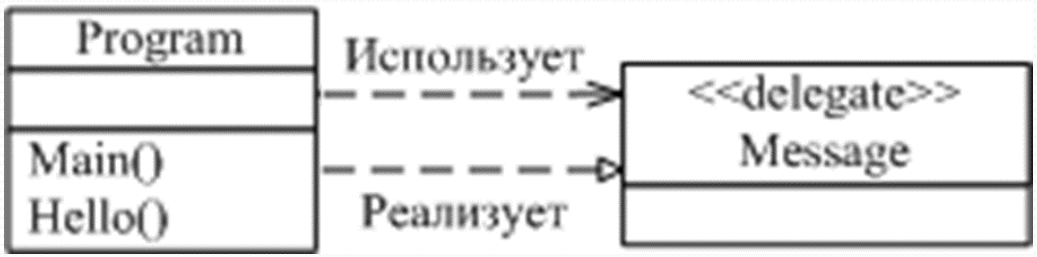
```
(int x, int y) => x + y;  // анонимное лямбда-выражение
int sum(x, y) => x + y;   // поименованное лямбда-выражение
```

Делегат(условно)

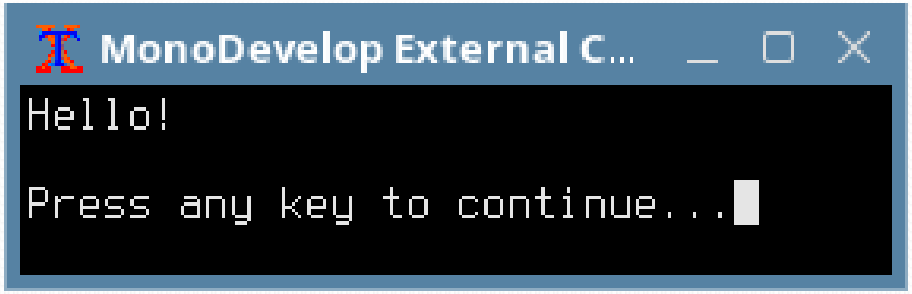


Пример делегата без параметров

Ex7.04




```
using System;
namespace Ex7_04{
class Program
{
    delegate void Message(); // объявление делегата
    static void Main()
    {
        Message mes;            // создание переменной - делегата
        mes = Program.Hello;    // присваивание адреса метода
        mes();                  // вызов метода
    }
    static void Hello() => Console.WriteLine("Hello!");
}
}
```



Описание метода лямбда-выражением

Пример делегата с параметрами

Ex7.05

```
using System;
namespace Ex7_05 {
class Program {
    delegate int Operation(int x, int y);
    static void Main()
    {
        Operation operation = new Operation(Program.Add);
        int result = operation(4,5);
        Console.WriteLine(result);

        operation = Program.Multiply;
        result = operation(4,5);
        Console.WriteLine(result);
    }
    static int Add(int x, int y) => x + y;
    static int Multiply(int x, int y) => x * y;
    }
}
```

Присваивание адреса лямбда-выражения Add

Вызов лямбда-выражения через делегата

Присваивание адреса лямбда-выражения Multiply

Вызов лямбда-выражения через делегата

Стандартные обобщенные делегаты

- делегат Действие – выполняет некоторую операцию:

```
public delegate void Action<in T>(T obj);
```

Пример:

```
Action<string> message;
message = s => Console.WriteLine(s);
message ("Hello, World!");
```

Перегружен для разных списков параметров

Перегружен для разных списков параметров

- делегат Функция – преобразует in в out :

```
public delegate TResult Func<in T,out TResult>(T arg);
```

Пример:

```
Func<string, string> selector = str => str.ToUpper();
string[] words = { "orange", "apple", "Article", "elephant" };
IEnumerable<String> aWords = words.Select(selector); // обрабатывает
                                                      //каждый элемент массива
```

- делегат Предикат – проверяет заданное условие:

```
public delegate bool Predicate<in T>(T obj);
```

(см. пример на следующем слайде)

Пример использования делегата Предикат

Метод Array.Find (поиск первого элемента, удовлетворяющего условию):

```
public static T? Find<T> (T[] array, Predicate<T> match);
```

Ex7.09

```
public class Point
{
   public int X,Y;
   public Point(int _x,int _y){ X = _x;  Y = _y; }
}
public class Example
{
   public static void Main()
   {
      Point[] points = { new Point(100, 200),
             new Point(150, 250), new Point(250, 375),
             new Point(275, 395), new Point(295, 450) };
      Point f = Array.Find(points,x => x.X * x.Y > 100000 );
      Console.WriteLine("Found: X = {0}, Y = {1}",f.X, f.Y);
   }
}
```

Условие поиска - делегат Predicate

Вариант 2

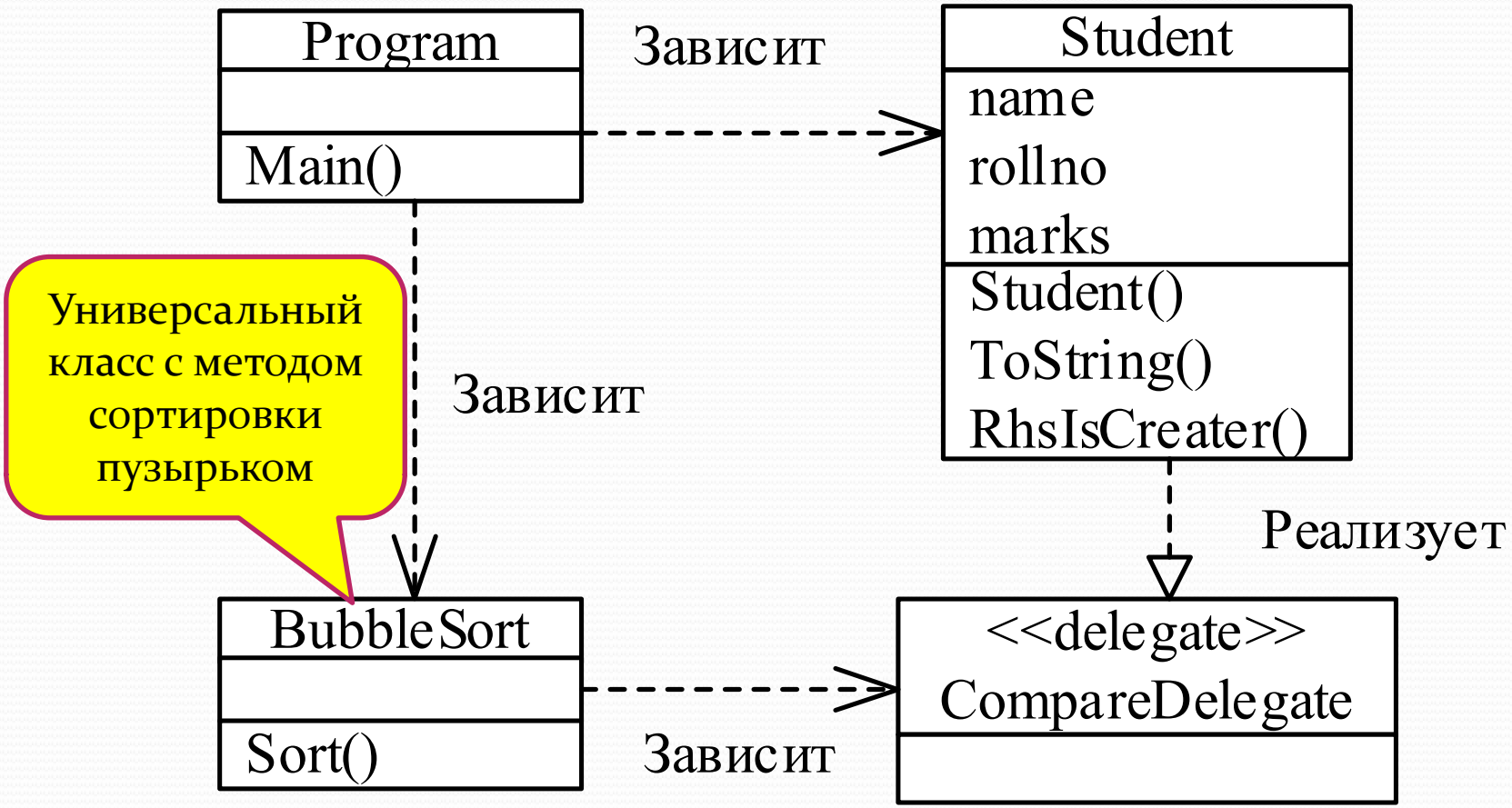
Ex7.09, a

```
public class Point
{
   public int X,Y;
   public Point(int _x,int _y){ X = _x;  Y = _y; }
}
public class Example
{
   public static void Main()
   {
      Point[] points = { new Point(100, 200),
             new Point(150, 250), new Point(250, 375),
             new Point(275, 395), new Point(295, 450) };
       // Predicate<Point> predicate = x => x.X * x.Y > 100000;
      static bool predicate(Point x) => x.X * x.Y > 100000;
      Point? f = Array.Find(points, predicate);
      Console.WriteLine("Found: X = {0}, Y = {1}",f.X, f.Y);
   }
}
```

Условие поиска - делегат Predicate

Пример использования делегата в качестве функционального параметра метода сортировки

Ex7.06



Пример (начало файла student.cs)

```
using System;
namespace Ex7_06
{
    class Student
    {
        string name;
        int rollno;
        int marks;
        public Student(string name,int rollno,int marks)
        {
            this.name = name; this.rollno = rollno;
            this.marks = marks;
        }
        public override string ToString()
        {
            return string.Format("Name => {0},
                   RollNumber => {1},
                   Marks => {2} ", name, rollno, marks);
        }
```

Пример (окончание файла student.cs)

```
        public static bool RhsIsGreater(object lhs,
                                        object rhs)
        {
            Student stdLhs = (Student)lhs;
            Student stdRhs = (Student)rhs;
            return stdRhs.marks > stdLhs.marks;
        }
    }
}
```

Пример (файл BubbleSort.cs)

Метод сортирует элементы типа Object

```
using System;
namespace Delegate{
   public class BubbleSort
   {
      static public void Sort(object[] sortArray,
                              Program.CompareDelegate gtMethod) {
         int j = 0;  bool k = true;
         int n = sortArray.Length;
         while (++j < n && k) {
            k = false;
            for (int i = 0; i < n-j; i++){
              if(gtMethod(sortArray[i],sortArray[i+1])){
                 k = true;
                 object temp = sortArray[i];
                 sortArray[i] = sortArray[i + 1];
                 sortArray[i + 1] = temp;
              }
            }
         }
      }
   }
}
```

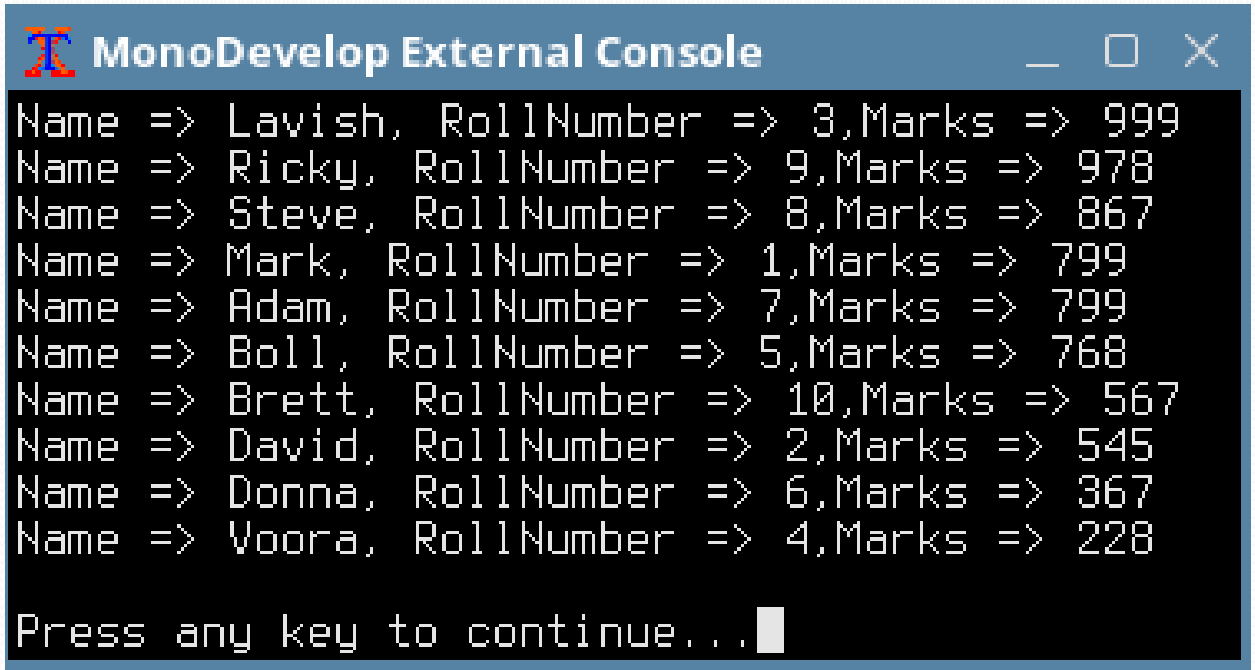
Пример (начало файла program.cs)

```
using System;
namespace Ex7_06
{
    public class Program
    {
        public delegate bool CompareDelegate(object lhs,
                                             object rhs);
        static void Main(string[] args)
        {
            Student[] students = {
            new Student("Mark", 1, 799),
            new Student("David", 2, 545),
            new Student("Lavish", 3, 999),
            new Student("Voora", 4, 228),
            new Student("Boll", 5, 768),
            new Student("Donna", 6, 367),
            new Student("Adam", 7, 799),
            new Student("Steve", 8, 867),
            new Student("Ricky", 9, 978),
            new Student("Brett", 10, 567)
            };
```

Пример (окончание файла program.cs)

```
        CompareDelegate StudentCompareOp = new
                CompareDelegate(Student.RhsIsGreater);

        BubbleSort.Sort(students, StudentCompareOp);
        for (int i = 0; i < students.Length; i++)
        {
            Console.WriteLine(students[i].ToString());
        }
    }
  }
}
```



7.7 События

Событие – элемент класса, позволяющий объектам класса посылать уведомление об изменении своего состояния. При этом активируются соответствующие обработчики событий.

Создание события:

1. Определить условие возникновения события и методы, которые должны сработать.
2. Определить сигнатуру этих методов и создать многоадресный делегат на основе этой сигнатуры.
3. Создать общедоступное событие на основе этого делегата и вызвать, когда условие сработает.
4. Обязательно (где угодно) подписать на это событие методы, которые должны сработать и сигнатуры которых подходят делегату.

Синтаксис описания события:

[**Атрибуты**][**Спецификаторы**] **`event`** **Тип Имя**

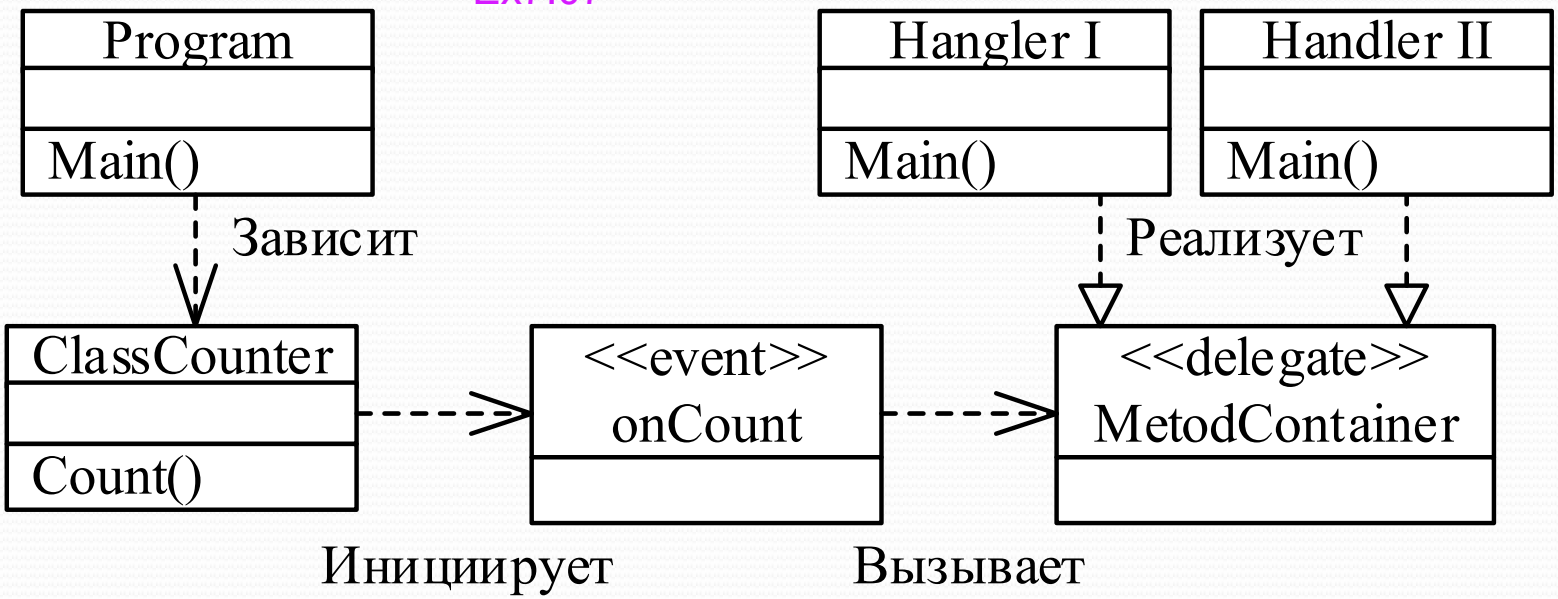
Спецификаторы: new, public, protected, internal, private, static, virtual, sealed, override, abstract, extern.

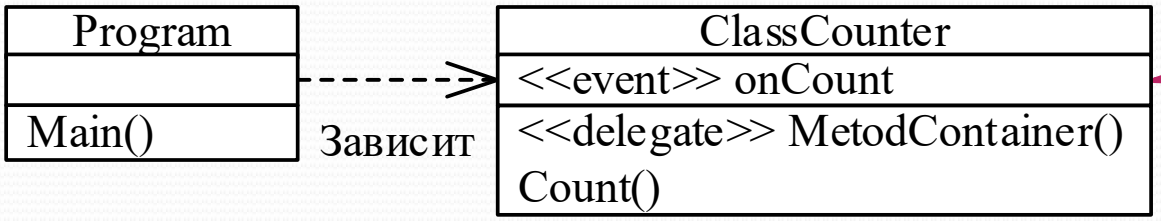
Тип совпадает с типом делегата.

Пример объявления и обработки события

Событие генерируется, когда счетчик достигнет значения 71. При обработке события выводятся два сообщения.

Ex7.07





Недетализированная схема

Пример

```
using System;
namespace Ex7_07
{
    class ClassCounter   // класс - инициатор события
    {
        public delegate void MethodContainer();  // делегат

        public event MethodContainer onCount;    // событие

        public void Count()
        {
            for (int i = 0; i < 100; i++)
            {
                if (i == 71)
                {
                    onCount();          // генерация события
                }
            }
        }
    }
```

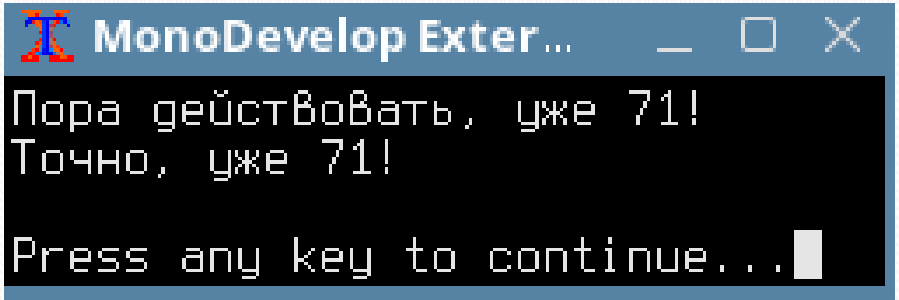
Многоадресный делегат

Пример (продолжение)

```
class Handler_I  // класс, содержащий обработчик события
    {
        public void Message()
        {
            Console.WriteLine("Пора действовать, уже 71!");
        }
    }
class Handler_II  // класс, содержащий обработчик события
    {
        public void Message()
        {
            Console.WriteLine("Точно, уже 71!");
        }
    }
```

Пример (окончание)

```
class Program
    {
        static void Main(string[] args)
        {
            ClassCounter Counter = new ClassCounter();
            Handler_I Handler1 = new Handler_I();
            Handler_II Handler2 = new Handler_II();
            // подписка на два события
            Counter.onCount += Handler1.Message;
            Counter.onCount += Handler2.Message;
             // вызов метода, который генерирует события
            Counter.Count();
        }
    }
}
```



7.8 Обобщения интерфейсов и их реализация

Возможно создание обобщений интерфейсов. Например, обобщение интерфейса для сравнения двух объектов определяется следующим образом:

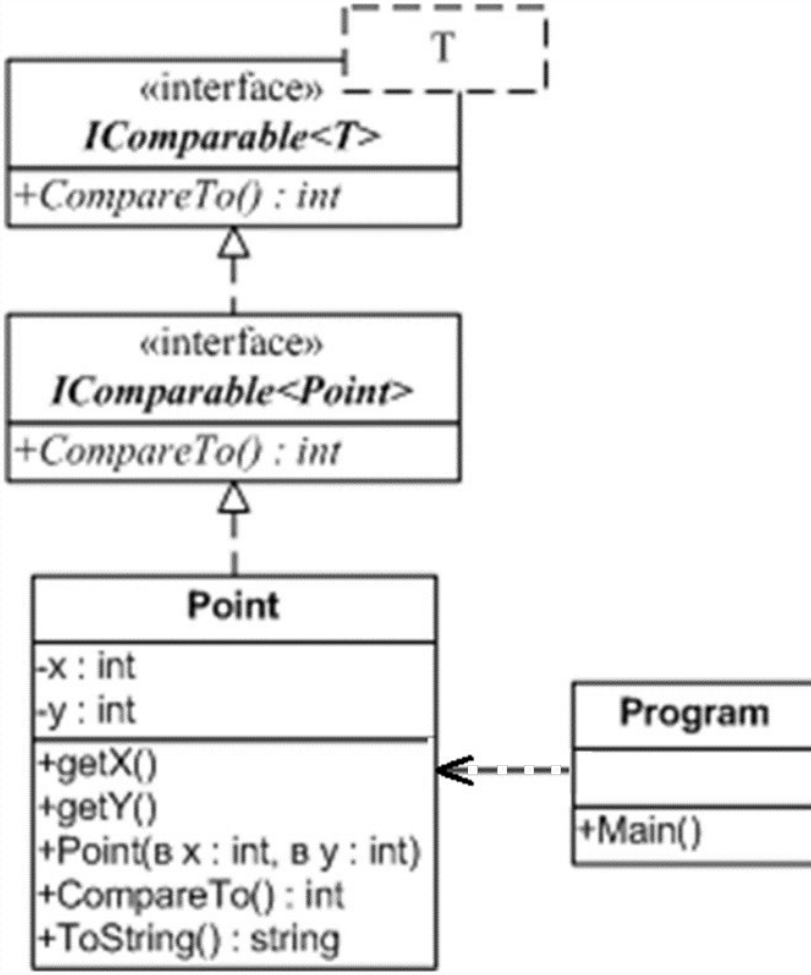
```
public interface IComparable<T>
{
    int CompareTo(T other);
}
```

Пример Сравнение векторов на плоскости по длине

Пример. Программа сравнения двух векторов на плоскости по длине. Метод, реализующий интерфейс должен быть объявлен **public.**

Ex7_08

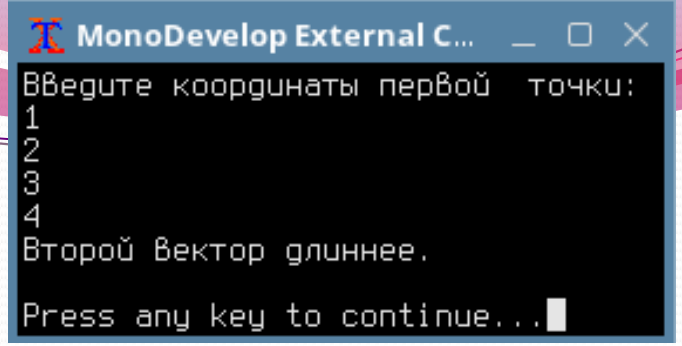
```
using System;
using System.Collections.Generic;
using static System.Math;
namespace Ex7_08
{
```



Пример (продолжение)

```
class Point:IComparable<Point>
{    double x, y;
     public double getX()
     {  return x; }
     public double getY()
     {  return y; }
     public Point(double _x, double _y)
     {  x = _x; y = _y;       }
     public int CompareTo(Point p)
     {  double l = Sqrt(x*x+y*y);
        double lp = Sqrt(p.x*p.x+p.y*p.y);
        if ( Abs(l-lp) < 1E-6) return 0;
        else if (l-lp >= 1E-6) return 1;
           else return -1;
     }
}
```

Пример (продолжение)




```
class Program
{ static void Main()
  { Console.WriteLine("Введите координаты первой  точки:");
    double x1 = int.Parse(Console.ReadLine());
    double y1 = int.Parse(Console.ReadLine());
    double x2 = int.Parse(Console.ReadLine());
    double y2 = int.Parse(Console.ReadLine());
    Point a = new Point(x1,y1);
    Point b = new Point(x2,y2);
    if (a.CompareTo(b) == 0)
        Console.WriteLine("Вектора равны.");
    else
      if (a.CompareTo(b) > 0)
          Console.WriteLine("Первый вектор длиннее.");
      else Console.WriteLine("Второй вектор длиннее.");
  }
}
}
```